

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3.6.1

UPGRADE TO 1.2

 Copy as Markdown

Upgrade to 1.2

Serverpod 1.2. is backward compatible with Serverpod 1.0 and Serverpod 1.1. There are a few changes to the database layer, meaning you probably want to use the new methods. The old methods still works, but have been deprecated and will be permanently removed with the upcoming version 2.

Database migrations are new in Serverpod 1.2. You can still opt to manage your database manually, but it is recommended that you move to the new migration system. Using the new migration will make keeping your database up-to-date much easier.

Updating your CLI

To update you Serverpod command line interface to the latest version, run:

```
dart pub global activate serverpod_cli
```

You can verify that you have the latest version installed by running:

```
serverpod version
```

Updating your pubspecs

To move to Serverpod 1.2, you will need to update the `pubspec.yaml` files of your `server`, `client`, and `flutter` directories. Anywhere `serverpod` is mentioned, change the version to `1.2.0` (or any later version of Serverpod 1.2). It is recommended to use explicit versions of the Serverpod packages, to make sure that they are all compatible.

Update to Dart 3



This update has bumped the minimum required dart version to `3.0.0`. You will have to change the Dart SDK version in all your `pubspec.yaml` files.

Old pubspec.yaml configuration:

```
...
environment:
  sdk: '≥2.19.0 <4.0.0'
```

Updated pubspec.yaml configuration:

```
...
environment:
  sdk: '≥3.0.0 <4.0.0'
```

The `Dockerfile` in your project should be updated with the new Dart version:

```
FROM dart:3.0 AS build

...
```

After updating your `pubspec.yaml` files, make sure to run `dart pub update` on all your packages. You must also run `serverpod generate` in your `server` directory.

Deprecated methods

In this version, we have completely reworked the database layer of Serverpod. The new methods have been placed under a static `db` field on the generated models. The old methods are still available, but the deprecation warnings will guide you toward moving to the updated API.

IMPORTANT

A few of the methods work slightly differently in their new versions. Most notably, the `insertRow` method will not modify the model you pass to it. Instead, it will return a modified copy with the inserted row `id`.

```
// The new find method is a drop-in replacement.
Example.find(...); // old
Example.db.find(...); // new
```

```
// The old findSingleRow method has changed name to findFirstRow but is otherwise a drop-in replacement.
Example.findSingleRow(...);
```

```

Example.db.findFirstRow(...);

// The new findById method is a drop-in replacement.
Example.findById(...); // old
Example.db.findById(...); // new

// The old delete method has been renamed to deleteWhere and now returns a list of ids of
rows that was deleted.
Example.delete(...);
Example.db.deleteWhere(...);

// The new findById method is a drop-in replacement but returns the id of the row
deleted.
Example.deleteRow(...); // old
Example.db.deleteRow(...); // new

// The old update method has been renamed too updateRow and now returns the entire
updated object as a new copy.
Example.update(...);
Example.db.updateRow(...);

// The old insert method has been renamed too insertRow. The object you pass in is no
longer modified, instead a new copy with the added row is returned which contains the
inserted id. This means no mutations of the input object.
Example.insert(...);
Example.db.insertRow(...);

// The new count method is a drop-in replacement.
Example.count(...);
Example.db.count(...);

```

Model changes

We have made some improvements to the Serverpod model files (previously referred to as protocol files or serializable entities). By default, the model files are now located in the `lib/src/models/` directory, although using `lib/src/protocol` still works.

When making the improvements to the model files, we made additions and changes to the syntax. All old keywords still work, but `serverpod generate` will give deprecation warnings, guiding you toward updating your models. The changes are listed below.

The keyword `api` has been deprecated and replaced with the new keyword `!persist` as a drop-in replacement.

Old syntax:

```

class: Example
table: example
fields:
  name: String

```

```
apiField: String, api
```

New syntax:

```
class: Example
table: example
fields:
  name: String
  apiField: String, !persist
```

The keyword **database** has been deprecated and replaced with the new keyword **scope** with the value **serverOnly** as a drop-in replacement.

Old syntax:

```
class: Example
table: example
fields:
  name: String
  serverField: String, database
```

New syntax:

```
class: Example
table: example
fields:
  name: String
  serverField: String, scope=serverOnly
```

The keyword **parent** has been moved and should be placed inside the new **relation** keyword, see the section on [relations](#) for the full new feature set.

Old syntax:

```
class: Example
table: example
fields:
  name: String
  parentId: int, parent=example
```

New Syntax:

```
class: Example
table: example
```

```
fields:
  name: String
  parentId: int, relation(parent=example)
```

Moved and renamed SQL file

Serverpod has moved and renamed the generated SQL file for the complete database schema. Instead of the file `generated/tables.pgsql`, Serverpod now includes it as a part of each migration located in the `migrations` directory, under the name `definition.sql`.

Initialize the migration system

If your project was created before migrations were introduced in Serverpod, you need to run a one-time setup to make it compatible with the new migration system. There are two guides to help you upgrade: one for projects that don't need to preserve any data and another for those that do.

The guides assume that you have already installed Serverpod 1.2 and that you have a project created with an earlier version of Serverpod.

No data to preserve

If it is not important to preserve the data that is in your database, you can simply remove the database and let the migration system create a new one.

1. Generate the project.

This ensures that the project is up to date with the latest version of Serverpod. Navigate to your project's `server` package directory and run the `generate` command.

```
$ serverpod generate
```

2. Create a migration for your project.

The migration system will create a migration as if the database needs to be initialized from scratch. Navigate to your project's `server` package directory and run the `create-migration` command.

```
$ serverpod create-migration
```

3. Recreate database.

In a Serverpod development project, the database is hosted in a docker container. To remove the existing database and start a new one run the following commands:

```
$ docker compose down -v  
$ docker compose up --build --detach
```

The command first removes the running container along with its volume and the second command starts a new database from scratch.

4. Initialize database from migration.

Initialize the database by applying the migration to it using the `--apply-migrations` flag when starting the server.

```
$ dart run bin/main.dart --apply-migrations
```

Data to preserve

If your project already has data in the database that should be preserved, we can use the repair migration system to bring the project up to date with the migration system.

1. Generate the project.

This ensures that the project is up to date with the latest version of Serverpod. Navigate to your project's `server` package directory and run the `generate` command.

```
$ serverpod generate
```

2. Create a migration for your project.

The migration system will create a migration as if the database needs to be initialized from scratch. Navigate to your project's `server` package directory and run the `create-migration` command.

```
$ serverpod create-migration
```

3. Create a repair migration.

The repair migration system will create a repair migration that makes your live database schema match the newly created migration. To enable the command to fetch your live database schema it requires a running server. Navigate to your project's `server` package directory and start the server, then run the `create-repair-migration` command.

```
$ dart run bin/main.dart  
$ serverpod create-repair-migration
```

INFO

When starting the server, warnings will be displayed about the database schema not matching the target database schema. These warnings are expected and can safely be ignored when creating the repair migration.

Use the `--mode` option to specify the database source to use. By default, the repair migration system connects to your `development` database using the information specified in your Serverpod config.

4. Apply the repair migration to your database.

To apply the repair migration to your database, restart the server using the `--apply-repair-migration` flag.

```
$ dart run bin/main.dart --apply-repair-migration
```

Closing remarks

Your project is now compatible with the database migration system in Serverpod 1.2.

Happy coding!